

ro.uvt.ri

Developer Onboarding & Workflow Guide

Headless WordPress + React Architecture

Purpose:

This document explains how new developers can install, run, understand, and contribute to the `ro.uvt.ri` project.

It covers environment setup, repository workflow, local WordPress usage, frontend development, Git collaboration, and troubleshooting.

Technology Stack

WordPress Headless CMS • REST API • React + Vite + TypeScript •
GitHub Workflow

Team Q-UW9

April 28, 2026

ro.uvt.ri

Developer Onboarding & Workflow Guide

Team Q-UW9

April 28, 2026

Contents

1	Project Overview	3
1.1	Repository Structure	3
2	Prerequisites	3
3	Initial Setup with Laragon First Time Only	4
3.1	Step 1 – Clone the Repository	4
3.2	Step 2 – Place the Project in Laragon	4
3.3	Step 3 – Start Laragon	4
3.4	Step 4 – Create the Database	4
3.5	Step 5 – Install or Configure WordPress	4
3.6	Step 6 – Log In to WordPress	5
3.7	Step 7 – Install Frontend Dependencies	5
3.8	Step 8 – Configure the API Connection	5
3.9	Step 9 – Start the React Development Server	5
4	Daily Workflow	6
4.1	Starting a Work Session	6
4.2	Working with WordPress Content	6
4.3	Creating a Feature Branch	6
4.4	Keeping Up to Date	7
4.5	Ending a Work Session	7
5	Alternative Setup: Docker Desktop and wp-env	7
5.1	Step 1 – Install wp-env	7
5.2	Step 2 – Start WordPress	8
5.3	Step 3 – Configure React for Docker WordPress	8
5.4	Stopping Docker WordPress	8
6	Useful Commands Reference	8
7	Troubleshooting	9
7.1	Laragon Site Does Not Open	9
7.2	Cannot Access WordPress Admin	9
7.3	Database Connection Error	9
7.4	React Cannot Fetch WordPress Data	9
7.5	CORS Errors When Fetching from WordPress	9
7.6	Permission Denied on Windows	10

7.7 Port Conflicts	10
8 Summary	10

1 Project Overview

This project follows a **Headless CMS** architecture:

- **Backend CMS:** WordPress, used for content management through the admin dashboard and exposed to the frontend through the REST API.
- **Local WordPress Environment:** Laragon is the recommended local development environment.
- **Frontend:** React, Vite, and TypeScript. This is the actual website that visitors interact with.

Tip

The main local workflow is:

Laragon → Start server → Create database → Open WordPress admin → Log in using lock.txt → Run React frontend → Work.

1.1 Repository Structure

```
ro.uvt.ro/  
|-- .gitignore          # Ignores WP core, node_modules, env files  
|-- README.md          # Quick-start instructions  
|-- lock.txt           # Local credentials / setup notes, not public  
|-- docs/              # This guide and other documentation  
|   +-- developer-guide.tex  
+-- frontend/          # React application (Vite + TypeScript)  
    |-- package.json  
    |-- src/  
    |   |-- App.tsx  
    |   |-- api/wordpress.js  
    |   +-- pages/  
    +-- vite.config.ts
```

Warning

The local WordPress installation is **not tracked by Git**. Each developer runs their own local WordPress instance through Laragon, while the React frontend code is tracked in the repository.

2 Prerequisites

Before cloning the repository, make sure the following tools are installed:

1. **Git** – <https://git-scm.com/downloads>
2. **Node.js** v18 or newer, LTS recommended – <https://nodejs.org/>
3. **Laragon** recommended local server stack for Windows – <https://laragon.org/>
4. **Docker Desktop** optional alternative for wp-env – <https://www.docker.com/products/docker-desktop/>

Tip

Use **Laragon** as the primary local WordPress environment. Docker Desktop is only an alternative if Laragon is not available.

3 Initial Setup with Laragon First Time Only

3.1 Step 1 – Clone the Repository

```
git clone https://github.com/Q-UW9/ro.uvt.ri.git
cd ro.uvt.ri
```

3.2 Step 2 – Place the Project in Laragon

The local WordPress installation should run inside Laragon's web root.

Typical path:

```
C:\laragon\www\ro.uvt.ri
```

If the repository was cloned somewhere else, either move it into `C:\laragon\www\` or clone it directly there:

```
cd C:\laragon\www
git clone https://github.com/Q-UW9/ro.uvt.ri.git
cd ro.uvt.ri
```

3.3 Step 3 – Start Laragon

Open Laragon and click:

```
Start All
```

This starts the local Apache/Nginx server and MySQL database server.

3.4 Step 4 – Create the Database

Open Laragon's database manager, such as HeidiSQL or phpMyAdmin, and create a new database:

```
ro_uvt_ri
```

Warning

Use the exact database name agreed by the team. If `lock.txt` contains a different database name, follow `lock.txt`.

3.5 Step 5 – Install or Configure WordPress

Open the local WordPress address:

```
http://ro.uvt.ri.test/wp-admin
```

If WordPress has not yet been installed, complete the installation wizard using the local database credentials provided by the team.

Typical Laragon database values are:

```
Database name: ro_uvt_ri
Username: root
Password:
Database host: localhost
```

Warning

Do not invent production credentials. For the correct local admin login, database details, and team-specific setup notes, use the credentials provided in `lock.txt`.

3.6 Step 6 – Log In to WordPress

Open:

```
http://ro.uvt.ri.test/wp-admin
```

Log in using the credentials provided in:

```
lock.txt
```

WordPress is used to manage content such as pages, posts, media files, documents, and any custom content types required by the project.

3.7 Step 7 – Install Frontend Dependencies

Move into the frontend folder and install dependencies:

```
cd frontend
npm install
```

3.8 Step 8 – Configure the API Connection

Create a `.env.local` file inside the `frontend/` directory:

```
# frontend/.env.local
VITE_WP_API_URL=http://ro.uvt.ri.test/wp-json
```

Warning

The `.env.local` file is **not tracked by Git**. Each developer must create their own local copy.

3.9 Step 9 – Start the React Development Server

From inside the `frontend/` directory, run:

```
npm run dev
```

The React application will usually start at:

```
http://localhost:5173
```

You can now work with both systems side by side:

- **WordPress admin:** `http://ro.uvt.ri.test/wp-admin`
- **React frontend:** `http://localhost:5173`

4 Daily Workflow

4.1 Starting a Work Session

```
# 1. Pull the latest code
git pull origin main

# 2. Open Laragon and click:
Start All

# 3. Open the WordPress admin
http://ro.uvt.ri.test/wp-admin

# 4. Log in using the credentials from lock.txt

# 5. Start the React frontend
cd frontend
npm run dev
```

4.2 Working with WordPress Content

Use the WordPress dashboard to manage backend content:

- create and edit pages;
- create and edit posts;
- upload media files and documents;
- manage categories and tags;
- manage custom post types if configured.

The React frontend reads this data through the WordPress REST API:

```
http://ro.uvt.ri.test/wp-json
```

Example endpoints:

```
http://ro.uvt.ri.test/wp-json/wp/v2/pages
http://ro.uvt.ri.test/wp-json/wp/v2/posts
http://ro.uvt.ri.test/wp-json/wp/v2/media
```

4.3 Creating a Feature Branch

Always work on a dedicated branch. Do not commit directly to `main`.

```
git checkout -b feature/your-feature-name

# Make your changes

git add .
git commit -m "feat: describe your change"
git push origin feature/your-feature-name
```

Then open a Pull Request on GitHub to merge into `main`.

4.4 Keeping Up to Date

Before starting new work, synchronize with the latest `main`:

```
git checkout main
git pull origin main

git checkout feature/your-feature-name
git rebase main
```

Tip

If new frontend dependencies were added by a teammate, run `npm install` again inside the `frontend/` directory.

4.5 Ending a Work Session

When finished:

```
# Stop the React dev server with:
Ctrl + C

# Then stop Laragon by clicking:
Stop
```

5 Alternative Setup: Docker Desktop and wp-env

Docker Desktop and `wp-env` can be used as an alternative local WordPress setup if Laragon is not available.

Warning

This is not the primary team workflow. Use this only if the project lead allows it or if Laragon cannot be used on your machine.

5.1 Step 1 – Install wp-env

Make sure Docker Desktop is running, then install `wp-env`:

```
npm -g install @wordpress/env
```


5.2 Step 2 – Start WordPress

From the project root:

```
wp-env start
```

This usually exposes WordPress at:

```
http://localhost:8888
http://localhost:8888/wp-admin
```

Default credentials are commonly:

```
Username: admin
Password: password
```

Warning

If team credentials are provided in `lock.txt`, use those instead.

5.3 Step 3 – Configure React for Docker WordPress

In `frontend/.env.local`, use:

```
VITE_WP_API_URL=http://localhost:8888/wp-json
```

Then start the React frontend:

```
cd frontend
npm run dev
```

5.4 Stopping Docker WordPress

```
wp-env stop
```

6 Useful Commands Reference

Command / Action	Description
Laragon: Start All	Start local web server and database
Laragon: Stop	Stop local web server and database
<code>http://ro.uvt.ro.test/wp-admin</code>	Open local WordPress dashboard
<code>cd frontend && npm install</code>	Install frontend dependencies
<code>cd frontend && npm run dev</code>	Start React dev server
<code>cd frontend && npm run build</code>	Build production bundle
<code>cd frontend && npm run lint</code>	Run ESLint checks
<code>git pull origin main</code>	Sync with latest remote changes
<code>git checkout -b feature/name</code>	Create a new feature branch
<code>git rebase main</code>	Rebase current branch onto main
<code>git stash</code>	Temporarily shelve uncommitted work
<code>wp-env start</code>	Alternative: start Docker WordPress
<code>wp-env stop</code>	Alternative: stop Docker WordPress
<code>wp-env clean all</code>	Alternative: reset Docker WordPress

7 Troubleshooting

7.1 Laragon Site Does Not Open

If `http://ro.uvt.ri.test` does not open:

- make sure Laragon is running;
- click **Start All**;
- check that the project folder is inside `C:\laragon\www`;
- restart Laragon;
- check that the generated local domain matches the folder name.

7.2 Cannot Access WordPress Admin

If `/wp-admin` does not work:

- confirm that WordPress is installed in the local project folder;
- confirm that the database exists;
- check `wp-config.php`;
- use the credentials from `lock.txt`.

7.3 Database Connection Error

Check that the database credentials match the local Laragon setup:

```
define( 'DB_NAME',      'ro_uvt_ri' );
define( 'DB_USER',      'root' );
define( 'DB_PASSWORD', '' );
define( 'DB_HOST',      'localhost' );
```

7.4 React Cannot Fetch WordPress Data

Check `frontend/.env.local`:

```
VITE_WP_API_URL=http://ro.uvt.ri.test/wp-json
```

Then restart the React dev server:

```
Ctrl + C
npm run dev
```

7.5 CORS Errors When Fetching from WordPress

If the frontend cannot request WordPress data due to CORS, configure the Vite proxy in `frontend/vite.config.ts`:

```
// vite.config.ts
export default defineConfig({
  server: {
    proxy: {
```

```
    '/wp-json': 'http://ro.uvt.ri.test'  
  }  
}  
)
```

Then fetch through the proxied path:

```
/wp-json/wp/v2/posts  
/wp-json/wp/v2/pages
```

7.6 Permission Denied on Windows

If Git reports permission or dubious ownership errors:

```
git config --global --add safe.directory 'C:/laragon/www/ro.uvt.ri'
```

7.7 Port Conflicts

If the React port is already in use, Vite will usually try another port automatically.

If Laragon ports conflict with another service, stop the conflicting service or change Laragon's Apache/Nginx/MySQL ports from Laragon settings.

8 Summary

**Clone repository → Start Laragon → Create database → Open
ro.uvt.ri.test/wp-admin → Log in with lock.txt → npm run dev → Start
working**

For questions or issues, open an issue on the GitHub repository:
<https://github.com/Q-UW9/ro.uvt.ri>